Fine-Tuning for Production

The Decision Framework · Dataset Construction · Data Quality · Training Specialists · The Ship Gate · Serving and Maintenance

WEEK 4 · DAY 27 **Saturday, August 1 2026** **Day 27 of 84**

DAYS 9 AND 10 TAUGHT YOU HOW. TODAY IS ABOUT WHETHER. Day 9 covered LoRA, rank, alpha, target modules, adapter merging. Day 10 covered full fine-tuning, LR schedules, gradient accumulation, FSDP, catastrophic forgetting. None of that is repeated today. The mechanics are maybe 10% of a production fine-tune — teams that skip straight to training are optimizing the part that was already solved.

The Three Questions That Actually Matter

```
1. SHOULD I fine-tune at all, or is this a prompting/RAG problem
   wearing a fine-tuning costume?

2. WHERE does the training data come from, and is it good enough?
   (This is 80% of the work and ~5% of the discussion in most teams.)

3. HOW DO I KNOW it is better — well enough to bet production on it?
```

Sec	Topic	Coverage
Sec 1	Decision Framework	Capability table, FineTuneDecision, four winning cases
Sec 2	Where Data Comes From	Four mining sources; composition with refusal set
Sec 3	Data Quality	Five filters, temporal holdout, contamination, PII
Sec 4	Training Specialists	FINE_TUNE_TARGETS priorities, mask_prompt_loss
Sec 5	The Ship Gate	Seven criteria, capability drift, two worked results
Sec 6	Serving + Maintenance	LoRA strategies, retraining cadence, provenance
Sec 7	FAANG Practice	Google/Microsoft/Amazon/Meta/Netflix tuning practice
Sec 8	Architect's Lens	Order of operations, data is the moat, shared budget
Sec 9	Exercises + Summary	Decision honesty, data mining, refusal set, train

Section 1 — The Decision Framework

Capability	Prompting	RAG	Fine-Tuning
Add knowledge	✗	✓✓✓	✗
Update knowledge	✗	✓✓✓	✗
Change FORMAT	✓✓	✗	✓✓✓
Change STYLE / TONE	✓✓	✗	✓✓✓
Teach a TASK	✓	✗	✓✓✓
Reduce latency	✗	✗	✓✓✓
Reduce cost	✗	✗	✓✓✓
Improve reasoning	✓✓	✗	✓

THE SINGLE MOST IMPORTANT ROW IS THE FIRST ONE. Fine-tuning does NOT reliably add knowledge. A model fine-tuned on your documentation does not 'know' your documentation — it learns the STYLE of it and then hallucinates content in that style, which is strictly WORSE than not knowing, because the output now looks authoritative. If the failing question is "what is our Q3 refund policy?" you have a RETRIEVAL problem, and fine-tuning will make the wrong answer sound more like your company wrote it.

```
@dataclass
class FineTuneDecision:
    """Run this BEFORE writing a training script. Most candidate problems
    do not survive it, and that is the point."""
    # Evidence gates — ALL must be TRUE
    prompting_exhausted:   bool   # tried CoT, few-shot, better instructions?
    rag_exhausted:         bool   # is retrieval actually the bottleneck?
    failure_is_consistent: bool   # same failure SHAPE, not random errors?
    have_1000_examples:    bool   # can you GET the data at all?
    have_holdout_eval:     bool   # can you MEASURE the improvement?
    # Economics
    monthly_volume:          int
    current_cost_per_query:  float
    expected_cost_per_query: float
    training_cost_usd:       float
    maintenance_hours_month: float = 8.0  # NOBODY budgets this; EVERYBODY pays it

    def verdict(self) -> dict:
        blockers = []
        if not self.prompting_exhausted:
            blockers.append("Prompting not exhausted — cheaper wins remain. "
                            "Day 21 ablation: CoT alone was worth +10.5pp.")
        if not self.rag_exhausted:
            blockers.append("Retrieval not exhausted — if the model lacks FACTS, "
                            "fine-tuning cannot supply them.")
        if not self.failure_is_consistent:
```

```
            blockers.append("Failures inconsistent — no learnable pattern; "
                            "you will train on NOISE.")
        if not self.have_1000_examples:
            blockers.append("Insufficient data — below ~1,000 quality examples, "
                            "few-shot prompting usually wins.")
        if not self.have_holdout_eval:
            blockers.append("No holdout eval — unable to tell improvement from "
                            "regression. NON-NEGOTIABLE.")
        monthly_saving = ((self.current_cost_per_query - self.expected_cost_per_query)
                          * self.monthly_volume)
        monthly_burden = self.maintenance_hours_month * 150.0   # engineer time
        net_monthly    = monthly_saving - monthly_burden
        payback_months = (self.training_cost_usd/net_monthly
                          if net_monthly > 0 else float("inf"))
        return {"proceed": not blockers and payback_months < 6,
                "blockers": blockers, "payback_months": round(payback_months,1),
                "note": "Payback beyond 6 months rarely survives contact with "
                        "reality — models change, requirements move, and the "
                        "fine-tune needs redoing before it pays for itself."}

# ■■ THE FOUR PROBLEMS WHERE FINE-TUNING GENUINELY WINS ■■■■■■■■■■■■■■■■■■
#  1. FORMAT / STRUCTURE ADHERENCE AT SCALE
#     Valid SQL, valid JSON, a fixed schema — EVERY time. A fine-tuned 8B
#     can beat a prompted 70B on structural compliance, because the format
#     becomes a property of the WEIGHTS rather than an instruction that
#     competes for attention.
#  2. COST AND LATENCY (the strongest business case)
#     Day 24 showed generation is the dominant cost. A fine-tuned 8B
#     matching a prompted 70B on YOUR narrow task is a STEP-CHANGE.
#  3. NARROW SPECIALIST BEHAVIOR
#     Day 20's RetrievalAgent does exactly one thing. Day 26's
#     VisualDescriber does exactly one thing. Your architecture is now
#     FULL of ideal fine-tuning targets.
#  4. IMPLICIT DOMAIN CONVENTIONS THAT RESIST INSTRUCTION
#     "In our schema, `dt` always means date-partitioned and must be
#     filtered." Explaining every convention in the prompt costs tokens on
#     EVERY request FOREVER; training it in costs ONCE.
```

Section 2 — Where the Data Comes From

THIS IS THE ACTUAL WORK. A mediocre training recipe on excellent data beats an excellent recipe on mediocre data, every single time. Your production system has been generating training data for weeks — Days 19, 21, 22 and 23 all wrote the exact records you need.

```
class ProductionDataMiner:
    """Four sources, in DESCENDING order of signal quality."""

    # ■■ SOURCE 1: VERIFIED SUCCESSES (the gold standard) ■■■■■■■■■■■■■■■■
    async def mine_verified_successes(self, tenant_id, since):
        """Day 19's AgentLongTermMemory already stores question->SQL pairs
        that succeeded. Day 22's user_feedback records thumbs-up. The
        INTERSECTION is your highest-quality data, and it ALREADY EXISTS."""
        rows = await self.db.fetch("""
            SELECT m.question, m.sql, a.artifact_versions, a.trace_id
            FROM agent_memory m JOIN audit_log a ON a.trace_id = m.trace_id
            WHERE m.tenant_id=$1 AND m.created_at >= $2
              AND a.payload->>'outcome' = 'success'
              AND (a.payload->>'validation_retries')::int = 0    -- FIRST TRY
              AND a.payload->>'user_feedback' = 'thumbs_up'""", tenant_id, since)

    # ■■ SOURCE 2: HUMAN CORRECTIONS (highest information density) ■■■■■■■
    async def mine_corrections(self, tenant_id):
        """A user who EDITED the generated SQL into a working query has
        handed you a labeled example at exactly the DECISION BOUNDARY where
        the model is wrong. ONE CORRECTION IS WORTH ROUGHLY TEN EASY
        SUCCESSES, because easy successes teach the model what it knows."""

    # ■■ SOURCE 3: TEACHER DISTILLATION (how you bootstrap) ■■■■■■■■■■■■■■
    async def distill_from_teacher(self, questions, schema_context):
        """Use the 70B to generate outputs, VERIFY BY EXECUTION, train the
        8B on what survived.
        THE VERIFICATION STEP IS THE ENTIRE VALUE. Distilling UNVERIFIED
        teacher output trains the student to reproduce the teacher's
        mistakes with MORE confidence and LESS capability."""
        for q in questions:
            result = await self.teacher.generate(q, schema_context)
            ok, rows, _ = await self.execute(result["sql"])       # Day 21
            if not ok: continue
            if not self.enforcer.check(result["sql"], schema_context)["is_faithful"]:
                continue
            out.append(TrainingExample(..., source="teacher_distilled",
                                       quality_score=0.85))

    # ■■ SOURCE 4: SYNTHETIC GENERATION (fills coverage gaps) ■■■■■■■■■■■■
    async def generate_synthetic(self, schema_ddl, target_patterns):
        """Generate questions for patterns production traffic UNDER-COVERS.
        Day 21's failure triage told you exactly what those are: nested set
        operations and HAVING-vs-WHERE placement.
        DIRECTION MATTERS: generate SQL FIRST, then ask for the natural
        language question. SQL->question is VERIFIABLE (you can execute the
        SQL); question->SQL is NOT."""

# ■■ THE DATASET COMPOSITION THAT ACTUALLY WORKS ■■■■■■■■■■■■■■■■■■■■■■■■
#   40%  verified production successes  — teaches the real distribution
```

```
#   25%  human corrections              — teaches the DECISION BOUNDARIES
#   20%  teacher-distilled + verified   — teaches breadth
#   10%  synthetic for known gaps       — teaches the long tail
#    5%  ADVERSARIAL / REFUSAL examples — teaches when NOT to answer
#
# THAT LAST 5% IS THE ONE EVERYONE OMITS.
#   If every training example is a successful answer, you train a model
#   that answers EVERYTHING — including questions it should DECLINE. You
#   will have fine-tuned away the ABSTENTION behavior Day 26 argued you
#   must have.
#
#   Include explicit examples of the CORRECT REFUSAL:
#     Q: "delete all the old orders"
#     A: "I can only generate SELECT queries. This request requires a write
#         operation, which I cannot perform."
#     Q: "show me the salary column"     (not in the retrieved schema)
#     A: "The retrieved schema does not contain a salary column. I cannot
#         answer this without a schema that includes it."
#
#   A MODEL THAT NEVER REFUSES IS NOT OBEDIENT. IT IS UNCALIBRATED.
```

Section 3 — Data Quality: The Part That Decides Everything

1,000 excellent examples beat 50,000 mediocre ones. The scoring and filtering pipeline is where a production fine-tune is won or lost.

```
class DatasetQualityPipeline:
    """Five filters in order. Expect to DISCARD 40-70% of raw mined data.
    If you are discarding under 20%, your filters are not working —
    production data is not that clean."""

    def build(self, raw) -> dict:
        # ■■ FILTER 1: DEDUPLICATION (near-duplicate, not exact) ■■■■■■■■
        deduped = self._dedupe_semantic(raw, threshold=0.95)
        # Production traffic is dominated by a few repeated questions.
        # Without dedup the model OVERFITS to "count all users" and gets
        # WORSE at everything else. This filter alone often removes 30%.

        # ■■ FILTER 2: CORRECTNESS VERIFICATION ■■■■■■■■■■■■■■■■■■■■■■■■■■
        verified = [ex for ex in deduped if self._verify_executes(ex)]
        # NEVER train on an output you have not verified. This is the
        # difference between DISTILLATION and COPYING MISTAKES.

        # ■■ FILTER 3: LENGTH AND COMPLEXITY DISTRIBUTION ■■■■■■■■■■■■■■■■
        balanced = self._balance_difficulty(verified)
        # Raw production traffic is ~70% easy. Train on that and the model
        # gets excellent at easy queries and NO BETTER at hard ones — the
        # opposite of what you needed the fine-tune for.

        # ■■ FILTER 4: CONTAMINATION CHECK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        clean = self._remove_eval_overlap(balanced)
        # If your training set contains your eval set, your eval reports a
        # number that MEANS NOTHING. Check by normalized question hash AND
        # by embedding similarity > 0.95 — PARAPHRASES LEAK TOO.

        # ■■ FILTER 5: PII AND SAFETY (Day 25) ■■■■■■■■■■■■■■■■■■■■■■■■■■■
        safe = [self._redact(ex) for ex in clean if not self._has_secrets(ex)]
        # TRAINING DATA BECOMES MODEL WEIGHTS. PII in the training set is
        # PII YOU CANNOT DELETE — right-to-erasure cannot reach into
        # weights. Not optional, and not reversible later.
        return {"dataset": safe, "retention_rate": len(safe)/max(len(raw),1)}

    def _balance_difficulty(self, examples):
        """TARGET distribution — deliberately HARDER than production:
             easy 25% / medium 35% / hard 30% / extra 10%
           Compare RAW production: easy 70% / medium 20% / hard 8% / extra 2%.
           YOU ARE TRAINING FOR THE FAILURES, NOT THE SUCCESSES."""

# ■■ THE HOLDOUT SPLIT (do this FIRST, before any filtering) ■■■■■■■■■■■■■
#   Split by TIME, not randomly:
#     train = examples before date T
#     eval  = examples after  date T
#
#   A RANDOM SPLIT LEAKS: the same question asked twice on the same day
#   lands in BOTH sets, and your eval score is inflated by memorization.
#   A temporal split also tests the thing you actually care about —
#   GENERALIZATION TO FUTURE TRAFFIC, which is the only traffic that
#   exists once you deploy.
```

Section 4 — Training the Specialists Your Architecture Needs

You are not fine-tuning "a model." You are fine-tuning specific components that Days 20, 24 and 26 already isolated for you.

Target	Candidate	Value	Risk	Pri
retrieval_agent	8B fine-tuned / classifier	Narrow, repetitive, structured. Easy win.	LOW	**1**
validation_agent	small classifier	Binary-ish; a 300M may match an 8B	LOW	2
generation_agent	8B fine-tuned on verified SQL	Largest cost line (Day 24). Highest prize.	HIGH	3
visual_describer	fine-tuned on your chart conventions	Retrieval bounded by description quality	MED	4

START AT PRIORITY 1. The retrieval agent is low-risk, high-volume and narrow — exactly the shape where fine-tuning reliably works. Proving the pipeline there earns you the right to attempt the generation agent.

```
@dataclass
class TrainingConfig:
    """Days 9-10 covered what these knobs DO. These are the PRODUCTION
    defaults that work for specialist task fine-tuning."""
    base_model:     str   = "meta-llama/Meta-Llama-3-8B-Instruct"
    method:         str   = "lora"    # start here; full FT only if LoRA plateaus
    lora_r:         int   = 16
    lora_alpha:     int   = 32
    target_modules: tuple = ("q_proj","k_proj","v_proj","o_proj",
                             "gate_proj","up_proj","down_proj")
    learning_rate:  float = 2e-4      # LoRA tolerates ~10x full-FT rates
    num_epochs:     int   = 3         # >3 on a small set MEMORIZES
    batch_size:     int   = 4
    grad_accum:     int   = 8         # effective batch 32
    warmup_ratio:   float = 0.03
    max_seq_len:    int   = 4096      # must fit schema context + output
    # PRODUCTION-CRITICAL, ROUTINELY OMITTED
    mask_prompt_loss: bool = True     # compute loss on the ANSWER ONLY

# ■■ WHY mask_prompt_loss MATTERS MORE THAN ANY HYPERPARAMETER ■■■■■■■■■■
#   If you compute loss over the WHOLE sequence, the model spends capacity
#   learning to predict your system prompt and schema context — text that
#   is IDENTICAL in every example and that it will NEVER need to generate.
#   Masking to the assistant turn only is often the difference between a
#   fine-tune that works and one that mysteriously does not.

# ■■ THE TRAINING RUN ITSELF IS THE EASY PART ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#   8B + LoRA + 5,000 examples + 3 epochs
#     ~ 2-4 hours on a single A100
#     ~ $8-15 of compute
#
#   Set that against ~3 WEEKS of data mining, cleaning, and eval-set
#   construction. THE RATIO IS THE LESSON. The GPU bill is a rounding
```

```
#   error next to the human time, which is why "we can't afford to
#   fine-tune" is almost never really about compute.
```

Section 5 — The Ship / No-Ship Decision

Day 23 built the golden-set gate for prompt changes. A fine-tuned model is a bigger change than any prompt edit, and it gets a stricter gate — seven criteria, any single failure blocks the ship.

```
@dataclass
class FineTuneGate:
    min_accuracy_gain_pp:   float = 0.0    # must not REGRESS vs baseline
    min_cost_reduction_pct: float = 40.0   # if not much cheaper, why bother?
    max_latency_pct:        float = 100.0  # must not be slower
    min_format_validity:    float = 0.99   # the thing FT is supposed to fix
    max_regression_cases:   int   = 0      # ZERO newly-broken golden cases
    min_refusal_accuracy:   float = 0.95   # still declines what it should
    max_catastrophic_drift: float = 0.05   # general capability retained

class FineTuneEvaluator:
    async def evaluate(self, student, baseline, gate) -> dict:
        # SAME eval set, SAME run (Day 23's rule)
        s = await CapstoneEvaluator(student).run(self.eval_set)     # Day 21
        b = await CapstoneEvaluator(baseline).run(self.eval_set)
        newly_broken = ({f["question"] for f in s.failures} -
                        {f["question"] for f in b.failures})
        # THE TEST NOBODY RUNS: did fine-tuning damage GENERAL capability?
        drift       = await self._capability_drift(student, baseline)
        # Did it FORGET HOW TO REFUSE? (Section 2's 5% adversarial set)
        refusal_acc = await self._refusal_accuracy(student)
        if refusal_acc < gate.min_refusal_accuracy:
            failures.append(f"Refusal accuracy {refusal_acc:.1%} — the model "
                            f"has been trained OUT OF DECLINING. SAFETY ISSUE.")
        if drift > gate.max_catastrophic_drift:
            failures.append(f"Capability drift {drift:.1%} — general ability "
                            f"damaged (catastrophic forgetting, Day 10)")

    async def _capability_drift(self, student, baseline) -> float:
        """Run a small GENERAL benchmark — NOT your task. A model
        fine-tuned hard on SQL can lose the ability to explain its
        reasoning, follow a new instruction, or handle an unusual request.
        Day 10 NAMED this; here is where you actually MEASURE it."""
```

```
■■■ RESULT A — RETRIEVAL AGENT FINE-TUNE (priority 1) ■■■
  Baseline: llama-3-8b prompted, 4-shot
  Student:  llama-3-8b + LoRA, 4,200 examples, 3 epochs

  Schema filtering F1     0.847 -> 0.912   +6.5pp   OK
  Format validity         0.943 -> 0.998   +5.5pp   OK
  Tokens per call         1,850 -> 640     -65%     OK  (no few-shot needed)
  p95 latency              410ms -> 260ms  -37%     OK
  Newly-broken golden cases                 0       OK
  Refusal accuracy        0.97  -> 0.96    -1pp     OK  (above floor)
  Capability drift                          2.1%    OK  (under 5%)
  -> SHIP

  THE TOKEN REDUCTION IS THE QUIET HEADLINE. The fine-tuned model no
  longer needs few-shot examples in the prompt, because THE EXAMPLES ARE
  IN THE WEIGHTS. That is a permanent 65% input reduction on every call
  to this agent, FOREVER — a bigger and more durable win than the
  accuracy gain.

■■■ RESULT B — GENERATION AGENT FINE-TUNE (priority 3) ■■■
```

```
Baseline: llama-3-70b prompted, CoT + dynamic few-shot
Student:  llama-3-8b + LoRA, 12,000 examples, 3 epochs

Execution accuracy     84.5% -> 79.8%   -4.7pp   FAIL (regression)
Cost per query       $0.0027 -> $0.0009 -67%     OK
p95 latency            3,740 -> 1,910ms -49%     OK
Newly-broken golden cases                14      FAIL
-> NO SHIP (as a full replacement)

BUT LOOK AT THE BREAKDOWN BY DIFFICULTY:
   easy    95.9% -> 95.1%   -0.8pp
   medium  88.5% -> 86.9%   -1.6pp
   hard    73.2% -> 61.0%  -12.2pp   <- where it ALL went
   extra   57.1% -> 38.4%  -18.7pp

THE CORRECT DECISION IS NOT "SHIP" OR "DON'T SHIP".
It is: ROUTE easy and medium to the fine-tuned 8B, keep the 70B for
hard and extra — which is exactly Day 24's ComplexityRouter, now with
a much stronger small tier.

Blended: 82.9% accuracy at $0.0016/query.
Against the SLO floor of 82% that clears — but it spends 1.6pp of the
0.8pp budget you had left after Day 24... WHICH MEANS IT DOES NOT FIT.

-> Honest conclusion: ship the retrieval fine-tune; route with the
   generation fine-tune ONLY after recovering accuracy headroom
   elsewhere. COST OPTIMIZATION AND FINE-TUNING DRAW ON THE SAME BUDGET.
```

Section 6 — Serving and Maintaining Fine-Tuned Models

A fine-tuned model is a NEW PRODUCTION DEPENDENCY with an ongoing cost that nobody puts in the proposal. A prompt change ships in seconds and rolls back in seconds (Day 23). A fine-tune takes days and rolls back to a MODEL, not a pointer. Prefer the prompt every time it can do the job.

```
class LoRAServingStrategy:
    """LoRA adapters can be HOT-SWAPPED at serving time — vLLM supports
    serving multiple adapters against one base model. This is what makes
    per-tenant specialization economically viable."""

    # ■■ OPTION A: MERGED WEIGHTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    #   Merge the adapter into the base; serve as a normal model.
    #   PRO: zero inference overhead
    #   CON: a full model copy per variant; cannot swap per request
    #   USE: one global specialist, high volume
    #
    # ■■ OPTION B: DYNAMIC ADAPTER LOADING ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    #   One base model in GPU memory, MANY adapters swapped per request.
    #   PRO: N tenants share one base; small memory per adapter
    #   CON: small per-request overhead
    #   USE: per-tenant fine-tunes — the case that justifies the whole
    #        approach for a multi-tenant product
    #
    # ■■ OPTION C: ADAPTER PER AGENT ROLE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    #   Day 20's three agents, three adapters, ONE shared 8B base.
    #   PRO: specialist quality at generalist infrastructure cost
    #   USE: THIS ARCHITECTURE, specifically

# ■■ THE MAINTENANCE COST NOBODY BUDGETS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#  A fine-tuned model must be RETRAINED when:
#    - the base model is upgraded          -> your adapter is tied to a version
#    - the schema/domain shifts materially -> Day 22's drift detectors fire
#    - accuracy decays on the canary set   -> Day 22 catches it, you must act
#    - the task definition changes         -> prompts edit in seconds;
#                                             WEIGHTS CANNOT
#
#  REALISTIC CADENCE: retrain quarterly, plus on any base model upgrade.
#  BUDGET: ~8 engineer-hours/month PER FINE-TUNED MODEL, ongoing, forever.
#
#  THIS IS WHY PRIORITY ORDER MATTERS. Each fine-tuned model is a PERMANENT
#  MAINTENANCE OBLIGATION. Three specialists is three obligations.
#  Fine-tune the FEWEST models that deliver the value.

@dataclass
class FineTunedModelArtifact:
    model_id:       str
    base_model:     str        # the adapter is MEANINGLESS without this
    base_model_sha: str
    adapter_sha:    str
    # DATASET PROVENANCE — the part Day 23 did not cover
    dataset_id:     str
    dataset_sha:    str
    dataset_size:   int
    training_config:dict
```

```
eval_results:   dict       # the gate run that AUTHORIZED the ship
trained_at:     datetime
approved_by:    str | None

# THE RULE: you must be able to REPRODUCE this artifact. If you cannot
# point to the exact dataset and config that produced these weights,
# you cannot debug a regression, satisfy an audit (Day 25), or retrain
# deliberately — YOU CAN ONLY RETRAIN AND HOPE.
```

Section 7 — FAANG Fine-Tuning Practice

Google — Supervised Tuning and Distillation on Vertex AI

Google exposes supervised fine-tuning for Gemini models on Vertex AI using parameter-efficient methods, and separately provides DISTILLATION tooling where a larger teacher generates training data for a smaller student. The documented guidance consistently frames tuning as appropriate for tasks with a clear OUTPUT FORMAT and a body of labeled examples, while directing KNOWLEDGE-oriented problems toward grounding and retrieval — the same first row of Section 1's capability table. The distillation product existing as a DISTINCT OFFERING is itself the signal: teacher-to-student transfer for a narrow task is common enough in production to warrant dedicated tooling, which is precisely the priority-1 pattern in Section 4.

Microsoft — Fine-Tuning in Azure OpenAI with Deployment Gating

Azure OpenAI supports fine-tuning with LoRA-based methods and REQUIRES a training and validation file split before a job will run, then produces a separately deployable model resource that must be EXPLICITLY DEPLOYED before it serves traffic. Two design choices are worth copying. First, the mandatory validation split makes it STRUCTURALLY IMPOSSIBLE to train without a holdout — the gate Section 1 refuses to proceed without. Second, separating 'the fine-tune finished' from 'the fine-tune serves traffic' mirrors Day 23's register-versus-promote split: training a model and approving it for production are DISTINCT ACTS with distinct owners.

Amazon — SageMaker JumpStart and Bedrock Custom Models

Amazon offers fine-tuning through SageMaker JumpStart for open models and Bedrock custom model creation for supported base models, with both producing artifacts that flow through the model registry and approval workflow discussed on Day 23. Bedrock additionally requires PROVISIONED THROUGHPUT for custom models, which surfaces the economic reality Section 5 quantifies: a custom model is NOT FREE TO SERVE, and the cost case must clear the dedicated-capacity commitment, not merely the per-token comparison. The practical consequence is that fine-tuning at LOW VOLUME frequently loses on economics even when it wins on accuracy.

Meta — Open Weights, Adapters, and Distilled Variants

Meta's release strategy makes fine-tuning practical for self-hosting teams by shipping open weights across MULTIPLE PARAMETER SIZES, and recent Llama releases have used distillation from larger models to produce smaller ones with disproportionately strong capability. Two consequences follow for your architecture. The adapter approach means a fine-tune does NOT FORK your entire serving stack — the base model stays shared, which is Section 6's Option C. And the existence of strong distilled small models RAISES THE BAR your own fine-tune must clear: before training a specialist, check whether an already-distilled smaller model in the same family solves the problem with NO TRAINING AT ALL.

Netflix — Personalization Models and Continuous Retraining

Netflix's published work on recommendation and personalization systems consistently emphasizes that a deployed model is NOT A FINISHED ARTIFACT — pipelines retrain on fresh interaction data ON A SCHEDULE, because the data distribution moves continuously and a model trained on last quarter's behavior degrades against this quarter's. The transferable discipline for LLM fine-tuning is Section 6's maintenance argument stated from EXPERIENCE rather than theory: the retraining cadence is part of the SYSTEM DESIGN, not an afterthought, and a team that cannot commit to retraining should not commit to fine-tuning.

Section 8 — Architect's Lens

```
THE ORDER OF OPERATIONS (violating this wastes MONTHS)

  1. Better prompting             hours    — Day 21: CoT alone was +10.5pp
  2. Better retrieval             days     — Day 21: RAG was worth +53.5pp
  3. Few-shot / dynamic examples  days     — Day 18
  4. A larger model               minutes  — often the honest answer
  5. FINE-TUNING                  weeks    — ONLY after 1-4 are exhausted

  FINE-TUNING IS FIFTH. It is the most interesting option and the one
  teams reach for FIRST, which is why so many fine-tuning projects produce
  a model that performs like a well-written prompt.

WHAT FINE-TUNING BUYS THAT NOTHING ELSE CAN
  NOT accuracy — prompting and retrieval usually get you further.
  What it UNIQUELY buys is COST AND LATENCY AT FIXED QUALITY: moving a
  capability from a 70B to an 8B, and moving few-shot examples OUT of the
  prompt and INTO the weights.
  Frame the business case that way and it survives review. Frame it as
  "the model will be smarter" and it will not.

THE TWO TESTS EVERYONE SKIPS
  1. CAPABILITY DRIFT — measure GENERAL ability, not just your task.
     A model fine-tuned hard on SQL can lose the ability to explain itself
     or follow a novel instruction.
  2. REFUSAL RETENTION — if 100% of training examples are successful
     answers, YOU HAVE TRAINED AWAY ABSTENTION. Day 26 argued abstention
     is a FEATURE; this is where you accidentally delete it.

DATA IS THE MOAT, NOT THE MODEL
  Anyone can run the same training script on the same base model. NOBODY
  ELSE HAS your verified production successes and your human corrections.
  The DATASET is the durable asset. Version it (Day 23), keep it PII-free
  (Day 25), and treat it as MORE VALUABLE than the weights it produced —
  because you can always retrain from data, but YOU CANNOT RECOVER DATA
  FROM WEIGHTS.

THE BUDGET INTERACTION NOBODY NOTICES UNTIL IT BITES
  Day 24 spent 1.7pp of a 2.5pp accuracy budget on cost optimization.
  A fine-tune that trades accuracy for cost draws on THE SAME ACCOUNT.
  Section 5's generation fine-tune looked shippable IN ISOLATION and did
  NOT FIT once the remaining 0.8pp was checked.
  Track ONE accuracy budget across cost work AND fine-tuning, or you will
  approve two changes that EACH fit and TOGETHER do not.

WHEN TO WALK AWAY MID-PROJECT
  - Retention rate after quality filtering under 30% -> your production
    data is too noisy; FIX THE UPSTREAM SYSTEM FIRST
  - The fine-tune MATCHES but does not BEAT few-shot -> ship the prompt;
    it has NO maintenance cost
  - You cannot construct a clean holdout -> STOP. You will be unable to
    tell success from self-deception, and you will ship on vibes.
```

Section 9 — Exercises and Summary

Exercise 1: Run the Decision Framework Honestly

```
# Fill in FineTuneDecision for the retrieval agent using YOUR numbers.
# Include maintenance_hours_month — DO NOT set it to zero.
# Report payback_months. If it exceeds 6, write one sentence explaining why
# you would proceed anyway, or accept that you would not.
# MOST CANDIDATE FINE-TUNES SHOULD FAIL THIS GATE. That is the gate working.
```

Exercise 2: Mine and Measure Your Data

```
# Run ProductionDataMiner against your own logs.
# Report the count from each of the FOUR sources.
# Then run DatasetQualityPipeline and report retention_rate.
# If retention is ABOVE 80%, your filters are TOO LOOSE — verify the dedup
# threshold and the contamination check specifically.
```

Exercise 3: Build the Refusal Set

```
# Write 50 adversarial examples where the CORRECT answer is a REFUSAL:
#   write operations, columns not in schema, cross-tenant requests,
#   ambiguous questions needing clarification.
# This is the 5% that keeps your model CALIBRATED.
# Then measure baseline refusal accuracy BEFORE training, so you have a
# number to compare against afterward.
```

Exercise 4: Fine-Tune the Priority-1 Target

```
# Train the retrieval agent: 8B + LoRA, mask_prompt_loss=True, 3 epochs.
# Run FineTuneEvaluator against the prompted baseline.
# Report ALL SEVEN gate criteria — including capability drift and refusal
# accuracy, not only accuracy and cost.
# Ship only on a CLEAN SWEEP.
```

Summary

Concept	Key Point
Days 9-10 vs today	They taught mechanics; today is whether, what data, does it ship
The capability table	Fine-tuning does NOT reliably add knowledge — that is retrieval
The costume mistake	"Fine-tune on our docs" makes wrong answers sound authoritative
FineTuneDecision	Five evidence gates plus payback; most candidates should fail
Maintenance in the model	~8 engineer-hours/month forever — the omitted cost
Four winning cases	Format adherence, cost/latency, narrow specialists, conventions
Data is the actual work	80% of effort, ~5% of most teams' discussion
Source 1	Verified production successes — Day 19's memory already stores them
Source 2	Human corrections — worth ~10x an easy success (decision boundary)
Source 3	Teacher distillation — the VERIFICATION step is the entire value

Source 4	Synthetic for gaps; generate SQL→question, not the reverse
Composition	40/25/20/10 plus **5% adversarial refusal**
The 5% everyone omits	Train only on successes and you delete abstention
Dedup	Production is repetitive; without it you overfit to "count users"
Difficulty rebalance	Production is 70% easy; train 25/35/30/10 — train for failures
Contamination check	Hash AND embedding similarity — paraphrases leak too
PII in training data	Becomes weights; right-to-erasure cannot reach it
Temporal holdout	Split by time, not randomly — tests future traffic
FINE_TUNE_TARGETS	Start at priority 1 (retrieval agent), not the generation agent
mask_prompt_loss	Loss on the answer only — often decides works vs doesn't
Training is the easy part	2–4 hours and $8–15 vs ~3 weeks of data work
FineTuneGate	Seven criteria; any single failure blocks the ship
Capability drift	Measure general ability — catastrophic forgetting, actually tested
Refusal retention	A model trained out of declining is a safety issue
The token headline	Few-shot moves into the weights — permanent 65% input cut
Route, don't replace	Generation FT failed as replacement, works as Day 24's small tier
The shared budget	Cost work and fine-tuning draw on the SAME accuracy account
LoRA serving	Option C — three adapters, one shared base, one per agent role
FineTunedModelArtifact	Base + adapter + DATASET provenance, or no reproducibility
Order of operations	Fine-tuning is FIFTH — after prompting, retrieval, few-shot, bigger model
What it uniquely buys	Cost and latency at fixed quality — not accuracy
Data is the moat	You can retrain from data; you cannot recover data from weights
When to walk away	Retention <30%, ties with few-shot, or no clean holdout

NEXT: Day 28 — Week 4 Capstone & Phase 1 Close

The final session before the pause. Consolidating Days 1–27 into one architecture, closing the open items carried since Day 21, and producing a handoff artifact you can pick up cold on 31 August 2026 when Week 5 resumes.